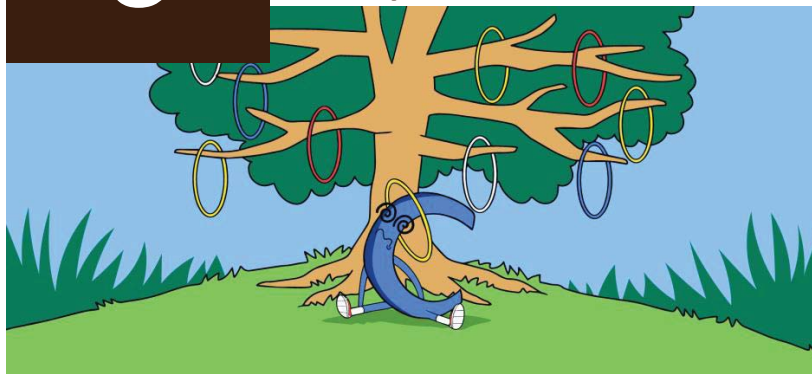


6

More Complex Repetitions



“Run through the hoops...”

x takes values from 1 to 10. For every x, y takes values from 0.5 to 2.25 in steps of 0.5 and for every y, z goes from 100 to 55 in decrements of -2, and for every z, m increases in multiples of 4 starting with 4. Lost in the hoops? Well, this chapter will show you how such complex repetitions can be implemented in C...

**Contents**

- The *for* Loop
 - Nesting of Loops
 - Multiple Initializations in the *for* Loop
- The *break* Statement
- The *continue* Statement
- The *do-while* Loop
- The Odd Loop
- Programs
- Exercises
- KanNotes

The programs in the last chapter showed how instructions in a program can be repeated using a **while** loop. This chapter explores the other two loops—**for** and **do-while**. There is more to looping than just repeating instructions. For example, what if we wish to terminate the loop abruptly, or skip some instructions as the loop executes, or repeat something infinite times, unknown number of times or only once. All these scenarios are handled in this chapter.

The *for* Loop

The **for** loop allows us to specify three things about the loop in a single line:

- (a) Setting a loop counter to an initial value.
- (b) Testing the loop counter to determine whether its value has reached the number of repetitions desired.
- (c) Increasing the value of loop counter each time the body of the loop has been executed.

The general form of **for** loop is as follows:

```
for ( initialize counter ; test counter ; increment counter )
{
    do this ;
    and this ;
}
```

Let us now write the simple interest program using **for**. Compare this program with the one that we wrote using **while** in Chapter 5.

```
/* Calculation of simple interest for 3 sets of p, n and r */
#include <stdio.h>
int main( )
{
    int p, n, count ;
    float r, si ;
    for ( count = 1 ; count <= 3 ; count = count + 1 )
    {
        printf ( "Enter values of p, n, and r " ) ;
        scanf ( "%d %d %f", &p, &n, &r ) ;
        si = p * n * r / 100 ;
        printf ( "Simple Interest = Rs.%f\n", si ) ;
    }
}
```

```
    return 0 ;  
}
```

You can observe that the three steps—initialization, testing and incrementation—required for the loop construct have now been incorporated in one single line in the **for** statement.

Let us now examine how the **for** statement gets executed:

- When **for** statement is executed for the first time, the value of **count** is set to an initial value 1.
- Next the condition **count <= 3** is tested. Since **count** is 1, the condition is satisfied and the body of the loop is executed for the first time.
- Upon reaching the closing brace of **for**, control is sent back to **for** statement, where the value of **count** gets incremented by 1.
- Again, the test is performed to check whether the new value of **count** is less than or equal to 3.
- If the value of **count** is less than or equal to 3, the statements within the braces of **for** are executed again.
- The body of **for** loop continues to get executed till **count** doesn't exceed the final value 3.
- When **count** reaches the value 4, the control exits from the loop and is transferred to the statement (if any) immediately after the body of **for**.

It is important to note that the initialization, testing and incrementation part of a **for** loop can be replaced by any valid expression. Thus, the following **for** loops are perfectly ok.

```
for ( i = 10 ; i ; i -- )  
    printf ( "%d ", i ) ;  
for ( i < 4 ; j = 5 ; j = 0 )  
    printf ( "%d ", i ) ;  
for ( i = 1 ; i <= 10 ; printf ( "%d ", i++ ) )  
    ;  
for ( scanf ( "%d", &i ) ; i <= 10 ; i++ )  
    printf ( "%d", i ) ;
```

Let us now write the program to print numbers from 1 to 10 in different ways. This time we would use a **for** loop instead of a **while** loop.

```
(a) #include <stdio.h>
int main( )
{
    int i;
    for ( i = 1 ; i <= 10 ; i = i + 1 )
        printf ( "%d\n", i );
    return 0 ;
}
```

Instead of **i = i + 1**, the statements **i++** or **i += 1** can also be used. Since there is only one statement in the body of the **for** loop, the pair of braces have been dropped. As with the **while**, the default scope of **for** is the immediately next statement after **for**.

```
(b) #include <stdio.h>
int main( )
{
    int i;
    for ( i = 1 ; i <= 10 ; )
    {
        printf ( "%d\n", i );
        i = i + 1 ;
    }
    return 0 ;
}
```

Here, the incrementation is done within the body of the **for** loop. In spite of this, the semicolon (;) after the condition is necessary.

```
(c) #include <stdio.h>
int main( )
{
    int i = 1 ;
    for ( ; i <= 10 ; i = i + 1 )
        printf ( "%d\n", i );
    return 0 ;
}
```

Here the initialization is done in the declaration statement itself, but still the semicolon before the condition is necessary.

```
(d) #include <stdio.h>
int main( )
{
    int i = 1 ;
    for ( ; i <= 10 ; )
    {
        printf ( "%d\n", i );
        i = i + 1 ;
    }
    return 0 ;
}
```

Here, neither the initialization nor the incrementation is done in the **for** statement, but still the two semicolons are necessary.

```
(e) #include <stdio.h>
int main( )
{
    int i ;
    for ( i = 0 ; i++ < 10 ; )
        printf ( "%d\n", i );
    return 0 ;
}
```

Here, the comparison as well as the incrementation is done through the same expression, **i++ < 10**. Since the **++** operator comes after **i**, comparison is done first, followed by incrementation. Note that it is necessary to initialize **i** to 0.

```
(f) #include <stdio.h>
int main( )
{
    int i ;
    for ( i = 0 ; ++i <= 10 ; )
        printf ( "%d\n", i );
    return 0 ;
}
```

Here again, both, the comparison and the incrementation are done through the same expression, **++i <= 10**. Since **++** precedes **i** firstly incrementation is done, followed by comparison. Note that it is necessary to initialize **i** to 0.

Nesting of Loops

The way **if** statements can be nested, similarly **whiles** and **fors** can also be nested. The following program shows how nested loops work.

```
/* Demonstration of nested loops */
#include <stdio.h>
int main( )
{
    int r, c, sum ;
    for ( r = 1 ; r <= 3 ; r++ ) /* outer loop */
    {
        for ( c = 1 ; c <= 2 ; c++ ) /* inner loop */
        {
            sum = r + c ;
            printf ( "r = %d c = %d sum = %d\n", r, c, sum ) ;
        }
    }
    return 0 ;
}
```

When you run this program, you will get the following output:

```
r = 1 c = 1 sum = 2
r = 1 c = 2 sum = 3
r = 2 c = 1 sum = 3
r = 2 c = 2 sum = 4
r = 3 c = 1 sum = 4
r = 3 c = 2 sum = 5
```

Here, for each value of **r**, the inner loop is cycled through twice, with the variable **c** taking values from 1 to 2. The inner loop terminates when the value of **c** exceeds 2, and the outer loop terminates when **r** exceeds 3.

As you can see, the body of the outer **for** loop is indented, and the body of the inner **for** loop is further indented. These multiple indentations make the program easier to understand.

Instead of using two statements, one to calculate **sum** and another to print it out, we can compact them into one single statement by saying:

```
printf ( "r = %d c = %d sum = %d\n", r, c, r + c ) ;
```

The way **for** loops have been nested here, similarly, two **while** loops can also be nested. Not only this, a **for** loop can occur within a **while** loop, or a **while** within a **for**.

Multiple Initializations in the *for* Loop

The initialization expression in the **for** loop can contain more than one statement separated by a comma. For example,

```
for ( i = 1, j = 2 ; j <= 10 ; j++ )
```

Multiple comma-separated incrementations can also be done in a **for** loop. Similarly, multiple conditions are allowed in the test expression. These conditions should be linked together using logical operators **&&** and/or **||** as shown below.

```
for ( i = 1, j = 2 ; j <= 10 && i <= 15 ; j++, i = i + 2 )
```

The *do-while* Loop

The **do-while** loop looks like this:

```
do
{
    this ;
    and this ;
} while ( this condition is true ) ;
```

There is a minor difference between the working of **while** and **do-while** loops. This difference lies in the place where the condition is tested. The **while** tests the condition before executing any of the statements within the **while** loop. As against this, the **do-while** tests the condition after having executed the statements within the loop.

This means that **do-while** would execute its statements at least once, even if the condition fails for the first time. The **while**, on the other hand will not execute its statements if the condition fails for the first time. This difference is brought about more clearly by the following program:

```
# include <stdio.h>
int main( )
{
    while ( 4 < 1 )
        printf ( " Credit is yours, pleasure is mine \n" ) ;
```

```
    return 0 ;  
}
```

Here, since the condition fails the first time itself, the **printf()** will not get executed at all. Let's now write the same program using a **do-while** loop.

```
#include <stdio.h>  
int main( )  
{  
    do  
    {  
        printf ( "Credit is yours, pleasure is mine\n" );  
    } while ( 4 < 1 );  
    return 0 ;  
}
```

In this program, the **printf()** would be executed once, since first the body of the loop is executed and then the condition is tested.

The *break* Statement

We often come across situations where we want to jump out of a loop instantly, without waiting to get back to the condition. The keyword **break** allows us to do this. When **break** is encountered inside a **while**, **for** or **do-while** loop, control automatically passes to the first statement after the loop. A **break** is usually associated with an **if**. Let's consider the following example to understand it:

Example 6.1: Write a program to determine whether a number is prime or not. A prime number is said to be prime if it is divisible only by 1 or itself.

All we have to do to test whether a number is prime or not, is to divide it successively by all numbers from 2 to one less than itself. If remainder of any of these divisions is zero, the number is not a prime. If no division yields a zero then the number is a prime number. Following program implements this logic:

```
#include <stdio.h>  
int main( )  
{  
    int num, i ;  
    printf ( "Enter a number " );
```

```
scanf ( "%d", &num ) ;
i = 2 ;
while ( i <= num - 1 )
{
    if ( num % i == 0 )
    {
        printf ( "Not a prime number\n" ) ;
        break ;
    }
    i++ ;
}
if ( i == num )
    printf ( "Prime number\n" ) ;
}
```

In this program, the moment **num % i** turns out to be zero, (i.e., **num** is exactly divisible by **i**), the message “Not a prime number” is printed and the control breaks out of the **while** loop. Why does the program require the **if** statement after the **while** loop at all? Well, there are two possibilities the control could have reached outside the **while** loop:

- (a) It jumped out because **num** turned out to be not a prime.
- (b) The loop came to an end because the value of **i** became equal to **num**.

When the loop terminates in the second case, it means that there was no number between 2 to **num - 1** that could exactly divide **num**. That is, **num** is indeed a prime. If this is true, the program should print out the message “Prime number”.

The keyword **break**, breaks the control only from the **while** in which it is placed. Consider the following program, which illustrates this fact:

```
# include <stdio.h>
int main( )
{
    int i = 1 , j = 1 ;
    while ( i++ <= 100 )
    {
        while ( j++ <= 200 )
        {
            if ( j == 150 )
                break ;
        }
    }
}
```

```
        else
            printf ( "%d %d\n", i, j ) ;
    }
}
return 0 ;
}
```

In this program when **j** equals 150, **break** takes the control outside the inner **while** only, since it is placed inside the inner **while**.

The *continue* Statement

In some programming situations, we want to take the control to the beginning of the loop, bypassing the statements inside the loop, which have not yet been executed. The keyword **continue** allows us to do this. When **continue** is encountered inside a loop, control jumps to the beginning of the loop for performing next iteration.

A **continue** is usually associated with an **if**. As an example, let's consider the following program:

```
# include <stdio.h>
int main( )
{
    int i, j ;
    for ( i = 1 ; i <= 2 ; i++ )
    {
        for ( j = 1 ; j <= 2 ; j++ )
        {
            if ( i == j )
                continue ;
            printf ( "%d %d\n", i, j ) ;
        }
    }
    return 0 ;
}
```

The output of the above program would be...

```
1 2
2 1
```

Note that when the value of **i** equals that of **j**, the **continue** statement takes the control to the incrementation part of the inner **for** loop bypassing the rest of the statements pending execution in the inner **for** loop.

A small quirk—when **continue** is encountered in a **while** or **do-while** control jumps to the condition, whereas in case of a **for** loop it takes the control to the incrementation part of the **for** loop.

The Odd Loop

The loops used so far executed the statements within them a finite number of times. However, at times one comes across a situation when it is not known beforehand, how many times the statements in the loop are to be executed. This situation can be programmed as shown below.

```
/* Execution of a loop an unknown number of times */
#include <stdio.h>
int main( )
{
    char another ;
    int num ;
    do
    {
        printf ( "Enter a number " ) ;
        scanf ( "%d", &num ) ;
        printf ( "square of %d is %d\n", num, num * num ) ;
        printf ( "Want to enter another number y/n " ) ;
        fflush ( stdin ) ;
        scanf ( "%c", &another ) ;
    } while ( another == 'y' ) ;
    return 0 ;
}
```

And here is the sample output...

```
Enter a number 5
square of 5 is 25
Want to enter another number y/n y
Enter a number 7
square of 7 is 49
Want to enter another number y/n n
```

In this program, the **do-while** loop would keep getting executed till the user continues to answer **y**. The moment user answers **n**, the loop terminates, since the condition (**another == 'y'**) fails. Note that this loop ensures that statements within it are executed at least once.

Though it is simpler to program such a requirement using a **do-while** loop, the same functionality, if required, can also be accomplished using **for** and **while** loops. You can try doing this as an exercise.

You must be wondering why have we used the function **fflush()**. The reason is to get rid of a peculiarity of **scanf()**. After supplying a number when we hit the Enter key, **scanf()** assigns the number to variable **num** and keeps the Enter key unread in the keyboard buffer. So when it's time to supply y or n for the question 'Want to enter another number (y/n)', **scanf()** will read the Enter key from the buffer thinking that user has entered the Enter key. To avoid this problem, we use the function **fflush()**. It removes or 'flushes out' any data remaining in the buffer. The argument to **fflush()** must be the buffer which we want to flush out. Here we have used 'stdin', which means buffer related with standard input device, i.e., keyboard.

P</> Programs

Problem 6.1

Write a program to print all prime numbers from 1 to 300.

Program

```
/* Generate all prime numbers from 1 to 300 */
#include <stdio.h>
int main( )
{
    int i, n = 1;
    printf ( "\nPrime numbers between 1 and 300 are :\n1\t" );
    for ( n = 1; n <= 300; n++ )
    {
        i = 2;
        for ( i = 2; i < n; i++ )
        {
            if ( n % i == 0 )
                break;
        }
    }
}
```

```
    }
    if ( i == n )
        printf ( "%d\t", n );
    }
    return 0 ;
}
```

Output

Prime numbers between 1 and 300 are :

1	2	3	5	7	11	13	17	19	23
29	31	37	41	43	47	53	59	61	67
71	73	79	83	89	97	101	103	107	109
113	127	131	137	139	149	151	157	163	167
173	179	181	191	193	197	199	211	223	227
229	233	239	241	251	257	263	269	271	277
281	283	293							

Problem 6.2

Write a program to add first seven terms of the following series using a **for** loop.

$$\frac{1}{1!} + \frac{2}{2!} + \frac{3}{3!} + \dots$$

Program

```
/* Sum of first seven terms of a series */
#include <stdio.h>
int main( )
{
    int i = 1, j ;
    float fact, sum = 0.0 ;
    for ( i = 1 ; i <= 7 ; i++ )
    {
        fact = 1.0 ;
        for ( j = 1 ; j <= i ; j++ )
            fact = fact * j ;
        sum = sum + i / fact ;
    }
}
```

```
printf ( "Sum of series = %f\n", sum ) ;  
return 0 ;  
}
```

Output

Sum of series = 2.718056

Problem 6.3

Write a program to generate all combinations of 1, 2 and 3 using for loop.

Program

```
/* Generate all possible combinations of 1 2 3 */  
# include <stdio.h>  
int main( )  
{  
    int i = 1, j = 1, k = 1 ;  
    for ( i = 1 ; i <= 3 ; i++ )  
    {  
        for ( j = 1 ; j <= 3 ; j++ )  
        {  
            for ( k = 1 ; k <= 3 ; k++ )  
                printf ( "%d %d %d\n", i , j , k ) ;  
        }  
    }  
    return 0 ;  
}
```

Output

```
1 1 1  
1 1 2  
..  
..  
2 1 1  
..  
..  
2 3 3
```

```
3 1 1
..
..
3 3 3
```

Exercises

[A] Answer the following questions:

(a) The **break** statement is used to exit from:

1. An **if** statement
2. A **for** loop
3. A program
4. The **main()** function

(b) A **do-while** loop is useful when we want that the statements within the loop must be executed:

1. Only once
2. At least once
3. More than once
4. None of the above

(c) In what sequence the initialization, testing and execution of body is done in a **do-while** loop?

1. Initialization, execution of body, testing
2. Execution of body, initialization, testing
3. Initialization, testing, execution of body
4. None of the above

(d) Which of the following is not an infinite loop?

- | | |
|--|---|
| 1. <pre>int i = 1 ; while (1) { i++ ; }</pre> | 2. <pre>for (; ;)</pre> |
| 3. <pre>int t = 0, f ; while (t) { f = 1 ;</pre> | 4. <pre>int y, x = 0 ; do { y = x ;</pre> |

- (e) Which of the following statements is true for the following program?

```
#include <stdio.h>
int main( )
{
    int x = 10, y = 100 % 90 ;
    for ( i = 1 ; i <= 10 ; i++ ) ;
        if ( x != y ) ;
            printf ( "x = %d y = %d\n", x, y ) ;
    return 0 ;
}
```

1. The **printf()** function is called 10 times.
 2. The program will produce the output `x = 10 y = 10`.
 3. The `;` after the **if (x != y)** will not produce an error.
 4. The program will not produce any output.
 5. The **printf()** function is called infinite times.
- (f) Which of the following statement is true about a **for** loop used in a C program?
1. **for** loop works faster than a **while** loop.
 2. All things that can be done using a **for** loop can also be done using a **while** loop.
 3. **for (; ;)** implements an infinite loop.
 4. **for** loop can be used if we want statements in a loop to get executed at least once.
 5. **for** loop works faster than a **do-while** loop.

[B] Attempt the following questions:

- (a) Write a program to print the multiplication table of the number entered by the user. The table should get displayed in the following form:

29 * 1 = 29

29 * 2 = 58

...

- (b) According to a study, the approximate level of intelligence of a person can be calculated using the following formula:

$$i = 2 + (y + 0.5x)$$

Write a program that will produce a table of values of **i**, **y** and **x**, where **y** varies from 1 to 6, and, for each value of **y**, **x** varies from 5.5 to 12.5 in steps of 0.5.

- (c) When interest compounds **q** times per year at an annual rate of **r** % for **n** years, the principal **p** compounds to an amount **a** as per the following formula

$$a = p (1 + r / q)^{nq}$$

Write a program to read 10 sets of **p**, **r**, **n** & **q** and calculate the corresponding **as**.

- (d) The natural logarithm can be approximated by the following series.

$$\frac{x-1}{x} + \frac{1}{2} \left(\frac{x-1}{x} \right)^2 + \frac{1}{2} \left(\frac{x-1}{x} \right)^3 + \frac{1}{2} \left(\frac{x-1}{x} \right)^4 + \dots$$

If **x** is input through the keyboard, write a program to calculate the sum of first seven terms of this series.

- (e) Write a program to generate all Pythagorean Triplets with side length less than or equal to 30.
- (f) Population of a town today is 100000. The population has increased steadily at the rate of 10% per year for last 10 years. Write a program to determine the population at the end of each year in the last decade.
- (g) Ramanujan number (1729) is the smallest number that can be expressed as sum of two cubes in two different ways—1729 can be expressed as $1^3 + 12^3$ and $9^3 + 10^3$. Write a program to print all such numbers up to a reasonable limit.
- (h) Write a program to print 24 hours of day with suitable suffixes like AM, PM, Noon and Midnight.
- (i) Write a program to produce the following output:

```

          1
        2   3
      4   5   6
    7   8   9  10
```



- 3 types of loops :
 - 1) while
 - 2) for
 - 3) do - while
- What can be done using one loop can always be done using the other two
- Usual usage :

while - to repeat something an unknown number of times, i.e. when we do not know at the time of writing the program how many times the loop is going to be executed

for - to repeat something a fixed number of times

do - while - to repeat something at least once
- Equivalent forms :

<pre>i = 1; while (i <= 10) { statement1; statement2; i++; }</pre>	<pre>for (i = 1 ; i <= 10 ; i++) { statement1; statement2; }</pre>	<pre>i = 1; do { statement1; statement2; i++; } while (i <= 10);</pre>
---	---	---
- for (; ;) is an infinite loop. while () results into an error
- Multiple initializations, conditions and incrementations in a for loop are acceptable. Ex. :


```
for ( i = 1 , j = 2 ; i <= 10 && j <= 24 ; i++ , j += 3 )
{
    statement1;
    statement2;
}
```

- *break* - terminates the execution of the loop
continue - abandons rest of the instructions in the loop and goes for the next iteration of the loop
- Usually *break* and *continue* are used in this form :

<pre>while (condition1) { if (condition2) break ; statement1 ; statement2 ; }</pre>	<pre>while (condition1) { if (condition2) continue ; statement1 ; statement2 ; }</pre>
---	--